

EXCEPTION HANDLING

Exception:

Mistakes in code that lead to abrupt stopping of program execution are called exceptions. (even if remaining lines of code are logically correct)

Mistakes/Problems that occur in program code are of two types.

1. Syntactical mistakes – Compiler throws Compile Time Errors (CTE)
2. Logical mistakes – Occurs in Run Time – JVM throws Exceptions

When?

Caught in compile time (checked exceptions) and run time (unchecked exceptions)

Where?

Exceptions occur,

- ❖ For a particular input
- ❖ At a particular statement

Exception Statements:

- Statement at which exception occurs – Exception Causing statement (ECS)
- Statement which are dependent – Exception Related Statement (ERS)

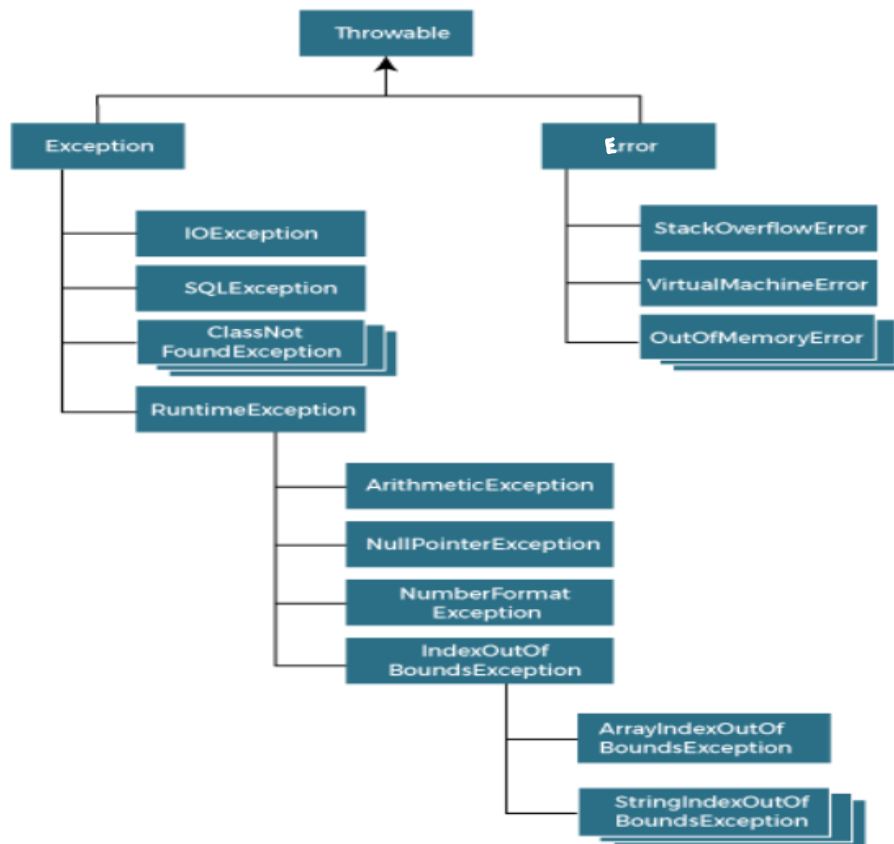
```
1  
2  
3 public class SampleDemo {  
4     public static void main(String[] args) {  
5         int a = 12;  
6         int b = 0;  
7         int res = a/b; //ECS  
8         System.out.println(res); //ERS  
9     }  
10 }  
11
```

Impact of Exception:

- ❖ Abrupt stopping of program execution skipping remaining lines of code even if they are logically correct
- ❖ Exception warning message is printed on the console in red coloured font
- ❖ Exception is thrown by Java Virtual Machine (JVM)

How are Exceptions stored?

Exceptions and Errors are in-built classes that belong to Throwable hierarchy as follows:



Methods to print Exception details:

1. void printStackTrace()

Prints the stack trace of the exception, including name, message and location of the error, which are required for debugging.

2. String getMessage()

Returns a short description of the exception. This method can be overridden in custom exception class to print suitable message.

TYPES OF EXCEPTIONS:

1. CHECKED EXCEPTIONS:

- Exceptions that are **compiler-aware** and are checked by the compiler during compilation.
- Checked exceptions inherit/extend Exception class.
- Compiler forces to handle checked exceptions immediately.
- If a new exception is created using throw keyword, using throws in method declaration is mandatory for checked exceptions.
- Custom(user-defined) checked exception classes can be created.
- Checked exceptions have to be explicitly caught or declared in method signature.
- Statements below throw statement become unreachable code.
 - ❖ **IOException**
 - ❖ **FileNotFoundException**
 - ❖ **ClassNotFoundException**
 - ❖ **InterruptedException**

2. UNCHECKED EXCEPTIONS:

- Exceptions that are **compiler-unaware** and are not checked by the compiler during compilation.
- Unchecked exceptions inherit/extend RuntimeException class.
- Compiler does not force to handle unchecked exceptions.
- If a new exception is created using throw keyword, using throws in method declaration is optional for unchecked exceptions.
- Custom(user-defined) un-checked exceptions can be created.
- Statements below throw statement become unreachable code.
 - ❖ **ArithmeticException**
 - ❖ **NullPointerException**
 - ❖ **ArrayIndexOutOfBoundsException**
 - ❖ **StringIndexOutOfBoundsException**
 - ❖ **ClassCastException**
 - ❖ **NumberFormatException**
 - ❖ **IllegalArgumentException**

DEALING WITH EXCEPTIONS:

There are two ways to deal with/overcome exceptions:

1. Throw the exception

throw keyword: for custom (user-created) exceptions

throws keyword: for checked and unchecked exceptions

2. Handle the exception

Using **try-catch** technique

DIFFERENCE BETWEEN throw AND throws:

Feature	throw	throws
Purpose	Explicitly throws an exception instance.	Declares exceptions a method might throw.
Usage	Followed by an exception object.	Followed by exception class names.
Placement	Within the method body.	In the method signature.
Quantity	Throws one exception at a time.	Can declare multiple exceptions.
Responsibility	Initiates an exception.	Shifts exception handling responsibility to the caller.

EXAMPLE FOR CHECKED EXCEPTION – INTERRUPTED EXCEPTION:

```
1 package exceptionhandling;
2
3 //interrupted exception is checked exception
4
5 public class Ck_InterruptedRuntimeException {
6     public static void main(String[] args) throws InterruptedException {
7         for(int i = 1; i<=5; i++) {
8             Thread.sleep(1000);
9             System.out.println(i);
10        }
11    }
12 }
13
```

<terminated>
1
2
3
4
5

In the above example, we have used a static method of in-built Thread class, called Thread.sleep(duration); We can specify a delay for each iteration of the loop.

But it throws a checked exception, compiler forces/compels to

- ❖ either add throws declaration, or
- ❖ surround with try-catch block

Hence, we have added throws declaration, and it will be implicitly handled.

Characteristics of try-catch technique:

1. 'try' block cannot be used alone, it should be followed by catch or finally block.
2. 'catch' block should have a proceeding try block.
3. Two finally blocks cannot be used in sequence, but can be used with different try blocks.
4. Exception, that is parent Exception in catch block should be used before finally block or as the last catch block.
5. When none of the catch blocks get matched, the parent Exception block gets executed, hence it has to be the last catch block.
6. 'finally' block is optional, but gets executed in all scenarios, whether the exception occurs or not.

SYNTAX OF TRY - CATCH BLOCKS:

```
try
{
    //statements;
}
catch (Throwable_type variable_name)
{
    //statements;
}
catch(Throwable_type variable_name)
{
    //statements;
}
finally
{
    //statements(always executed);
}
```

Workflow of try:

- ❖ 'try' is a keyword.
- ❖ If exception occurs in statements inside try, then JVM creates a memory block(object) in heap area.
- ❖ It returns the reference. This reference is thrown to catch blocks and concerned catch block variable gets that address.
- ❖ That reference variable can be used to access non-static methods of Exception class.
- ❖ 'new' keyword creates objects of any type. JVM creates objects only in Throwable hierarchy and hence are called Throwable type variables.

Three cases of try-catch technique:

1. try-catch block is present, but exception did not occur – try block gets executed
2. try-catch block is present, but exception occurred – catch block gets executed
3. try-catch block is present, but concerned catch block is absent – exception occurs

CASE 1: EXCEPTION DID NOT OCCUR

```
1 package exceptionhandling;
2
3 import java.util.Scanner;
4
5 public class TryCatch {
6     int num;
7     public static void main(String[] args) {
8         Scanner scn = new Scanner(System.in);
9         System.out.println("enter value of first number");
10        int a = scn.nextInt();
11        System.out.println("enter value of second number");
12        int b = scn.nextInt();
13        try {
14            int res = a/b;
15            System.out.println(res);
16        } // TryCatch tc = new TryCatch();
17        // tc = null;
18        // int ans = tc.num; // EXCEPTION CAUSING STATEMENT
19        // System.out.println(ans); // EXCEPTION RELATED STATEMENT
20    }
21    catch(ArithmeticException ae) {
22        System.out.println("Arithmetic Exception occurred");
23    }
24    catch(NullPointerException npe) {
25        System.out.println("NullPointerException occurred");
26    }
27    catch(Exception e) {
28        System.out.println("Exception occurred");
29    }
30    finally {
31        System.out.println("Thank you");
32    }
33    scn.close();
34 }
35 }
36 }
```

```
<terminated> TryCatch [Java Application] C:\Users\ne
enter value of first number
10
enter value of second number
5
2
Thank you
```

CASE 2: EXCEPTION OCCURED

```
1 package exceptionhandling;
2
3 import java.util.Scanner;
4
5 public class TryCatch {
6     int num;
7     public static void main(String[] args) {
8         Scanner scn = new Scanner(System.in);
9         System.out.println("enter value of first number");
10        int a = scn.nextInt();
11        System.out.println("enter value of second number");
12        int b = scn.nextInt();
13        try {
14            int res = a/b; // EXCEPTION CAUSING STATEMENT
15            System.out.println(res); // EXCEPTION RELATED STATEMENT
16        } // TryCatch tc = new TryCatch();
17        // tc = null;
18        // int ans = tc.num;
19        // System.out.println(ans);
20    }
21    catch(ArithmeticException ae) {
22        System.out.println("Arithmetic Exception occurred");
23    }
24    catch(NullPointerException npe) {
25        System.out.println("NullPointerException occurred");
26    }
27    catch(Exception e) {
28        System.out.println("Exception occurred");
29    }
30    finally {
31        System.out.println("Thank you");
32    }
33    scn.close();
34 }
35 }
36 }
```

```
<terminated> TryCatch [Java Application] C:\Users\ne
enter value of first number
10
enter value of second number
0
Arithmetic Exception occurred
Thank you
```

CASE 3: CONCERNED CATCH BLOCK IS ABSENT

```
1 package exceptionhandling;
2
3 import java.util.Scanner;
4
5 public class TryCatch {
6     int num;
7     public static void main(String[] args) {
8         Scanner scn = new Scanner(System.in);
9         System.out.println("enter value of first number");
10        int a = scn.nextInt();
11        System.out.println("enter value of second number");
12        int b = scn.nextInt();
13        try {
14            int res = a/b; // EXCEPTION CAUSING STATEMENT
15            System.out.println(res); // EXCEPTION RELATED STATEMENT
16            // TryCatch tc = new TryCatch();
17            // tc = null;
18            // int ans = tc.num;
19            // System.out.println(ans);
20        }
21        catch(ArithmeticException ae) {
22            // System.out.println("Arithmetic Exception occurred");
23        }
24        catch(NullPointerException npe) {
25            System.out.println("NullPointerException occurred");
26        }
27        catch(Exception e) {
28            System.out.println("Exception occurred");
29        }
30        finally {
31            System.out.println("Thank you");
32        }
33        scn.close();
34    }
35 }
36
```

```
<terminated> TryCatch [Java Application] C:\Users\vr
enter value of first number
10
enter value of second number
0
Exception occurred
Thank you
```

EXAMPLE FOR UNCHECKED EXCEPTION – ARITHMETIC EXCEPTION:

```
1 package exceptionhandling;
2
3 import java.util.Scanner;
4
5 public class Uck_ArithmeticExceptionEg {
6     public static void main(String[] args) {
7         double x = -10;
8         double y = 0;
9         System.out.println(x/y);
10        Scanner sn = new Scanner(System.in);
11        System.out.println("enter value of first number");
12        int a = sn.nextInt();
13        System.out.println("enter value of second number");
14        int b = sn.nextInt();
15        try {
16            int result = a/b;
17            System.out.println(result);
18        } catch(ArithmeticException ae) {
19            System.out.println("Arithmetic Exception occurred");
20            System.out.println("REASON FOR EXCEPTION: " + ae.getMessage());
21        }
22        sn.close();
23    }
24 }
25
```

```
<terminated> Uck_ArithmeticExceptionEg [Java Applic
-Infinity
enter value of first number
12
enter value of second number
0
Arithmetic Exception occurred
REASON FOR EXCEPTION: / by zero
```

EXAMPLE FOR UNCHECKED EXCEPTION – NULL POINTER EXCEPTION:

```
1 package exceptionhandling;
2
3 public class Uck_NullPointerExceptionEg {
4     static String name;
5     int number;
6     public static void main(String[] args) {
7         try {
8             System.out.println("value of static variable name: " + name);
9             System.out.println(name.length());
10        } catch (NullPointerException npe1) {
11            System.out.println("NPE occurred for string length");
12            System.out.println("REASON: " + npe1.getMessage());
13        }
14        UncheckedExceptionsEg uc = new UncheckedExceptionsEg();
15        uc = null;
16        try {
17            System.out.println(uc.number);
18        } catch (NullPointerException npe2) {
19            System.out.println("NPE occurred for null object reference");
20            System.out.println("REASON: " + npe2.getMessage());
21        }
22    }
23 }
```

```
<terminated> Uck_NullPointerExceptionEg [Java Application] C:\Users\naray.p2\pool\plugins\or
value of static variable name: null
NPE occurred for string length
REASON: Cannot invoke "String.length()" because "exception
NPE occurred for null object reference
REASON: Cannot read field "number" because "uc" is null
```

EXAMPLE FOR UNCHECKED EXCEPTION – ARRAY INDEX OUT OF BOUNDS EXCEPTION:

```
1 package exceptionhandling;
2
3 public class Uck_AIOOBEEg {
4     public static void main(String[] args) {
5         int arr[] = {12, 42, 56};
6         try {
7             System.out.println(arr[5]);
8         } catch (ArrayIndexOutOfBoundsException aiob) {
9             System.out.println("ArrayIndexOutOfBoundsException occurred");
10            System.out.println("REASON: " + aiob.getMessage());
11        }
12    }
13 }
14
```

```
<terminated> Uck_AIOOBEEg [Java Application] C:\Users\naray.p2\pool\p
ArrayIndexOutOfBoundsException occurred
REASON: Index 5 out of bounds for length 3
```

EXAMPLE FOR UNCHECKED EXCEPTION – STRING INDEX OUT OF BOUNDS EXCEPTION:

```
1 package exceptionhandling;
2
3 public class Uck_SIOOBEEg {
4     public static void main(String[] args) {
5         String str = "welcome";
6         try {
7             System.out.println(str.charAt(12));
8         } catch (StringIndexOutOfBoundsException siob) {
9             System.out.println("StringIndexOutOfBoundsException occurred");
10            System.out.println("REASON: " + siob.getMessage());
11        }
12    }
13 }
14
```

```
<terminated> Uck_SIOOBEEg [Java Application] C:\Users\naray.p2\pool\pl
StringIndexOutOfBoundsException occurred
REASON: Index 12 out of bounds for length 7
```

EXAMPLE FOR UNCHECKED EXCEPTION – CLASS CAST EXCEPTION:

```
1 package exceptionhandling;
2
3 class Animals8{}
4 class Dogs8 extends Animals8{}
5 class Cats8 extends Animals8{}
6
7 public class Uck_ClassCastExceptionEg {
8     public static void main(String[] args) {
9         Animals8 ad = new Dogs8();
10        try {
11            Cats8 c = (Cats8)ad;
12        } catch (ClassCastException cce) {
13            System.out.println("ClassCast Exception occurred");
14            System.out.println("REASON: " + cce.getMessage());
15        }
16    }
17 }
18
```

```
<terminated> Uck_ClassCastExceptionEg [Java Application] C:\Users\naray.p2\pool\plugins\org.eclipse.justi.open
ClassCast Exception occurred
REASON: class exceptionhandling.Dogs8 cannot be cast to class excepti
```


CUSTOM UNCHECKED EXCEPTION CLASS:

```
30 //CUSTOM UNCHECKED EXCEPTION CLASS
31 class ZeroFundsException extends RuntimeException
32 {
33     double extra;
34     double remaining;
35     ZeroFundsException(double extra, double remaining){
36         this.extra = extra;
37         this.remaining = remaining;
38     }
39     public String getMessage() // OVER RIDES GETMESSAGE METHOD
40     {
41         return "SORRY, you are short of: " + extra + " you have only: " + remaining;
42     }
43 }
44
45 class CheckAccounts{
46     double balance;
47     public void deposit(double amount) {
48         System.out.println("initial amount deposited: " + amount);
49         balance = balance + amount;
50     }
51     public void withdraw(double amount) // throws is optional
52     {
53         if(amount<=balance) {
54             System.out.println("withdrawn amount: " + amount);
55             balance = balance - amount;
56             System.out.println("remaining balance: " + balance);
57         }
58         else {
59             double excess = amount - balance;
60             throw new ZeroFundsException(excess, balance);
61         }
62     }
63 }
64
```

```
1 package exceptionhandling;
2 import java.util.Scanner;
3
4 public class CustomUck_Class {
5     public static void main(String[] args){
6         Scanner sn = new Scanner(System.in);
7         System.out.println("enter the initial amount to deposit");
8         double dep = sn.nextDouble();
9         CheckAccounts cn = new CheckAccounts();
10        cn.deposit(dep);
11        try
12        {
13            while(true)
14            {
15                System.out.println("*****enter amount to withdraw");
16                double amt = sn.nextDouble();
17                cn.withdraw(amt);
18            }
19        }
20        catch(ZeroFundsException e)
21        {
22            System.out.println("****over-ride getMessage() of Throwable class****");
23            System.out.println(e.getMessage());
24            e.printStackTrace();
25        }
26        sn.close();
27    }
28 }
29
```

<terminated> CustomUck_Class [Java Application] C:\Users\naray\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w

enter the initial amount to deposit

2000

initial amount deposited: 2000.0

*****enter amount to withdraw

1200

withdrawn amount: 1200.0

remaining balance: 800.0

*****enter amount to withdraw

1100

****over-ride getMessage() of Throwable class****

SORRY, you are short of: 300.0 you have only: 800.0

exceptionhandling.ZeroFundsException: SORRY, you are short of: 300.0 you have only: 800.0

at javaBTM/exceptionhandling.CheckAccounts.withdraw(CustomUck_Class.java:60)

at javaBTM/exceptionhandling.CustomUck_Class.main(CustomUck_Class.java:17)